DevOps Troubleshooting — Part 3

HashiCorp Vault

1. Application is unable to retrieve secrets from Vault. What will you check?
Answer:
First, I check whether the application can reach the Vault server and whether Vault is healthy. Then I verify the authentication method, role, token, policy, and requested secret path.
I also check whether the application identity has permission to read that specific secret. After fixing the authentication or policy issue, I test secret retrieval again.
Simple ga gurthupettuko:
Vault secret fail → Vault health → connectivity → authentication → role/policy → secret path → test.
Eng: I check Vault connectivity, authentication, policy permissions, and the secret path before testing retrieval again.
Example:
vault status → Vault healthy/sealed state check cheyyi.
vault login → authentication test cheyyi.
vault token lookup → token details/validity chudu.
vault policy read <policy-name> → policy permissions inspect cheyyi.
vault kv get <secret-path> → secret access test cheyyi.
Permission/auth issue fix chesi → application secret retrieval verify cheyyi.

---

2. Vault authentication suddenly started failing for an application. What will you investigate?
Answer:
First, I identify the authentication method being used, such as Kubernetes, AppRole, AWS IAM, or token authentication. Then I check whether the credentials, role mapping, token, or authentication configuration changed recently.
I also check Vault audit logs and application logs to identify the exact authentication failure.
Simple ga gurthupettuko:
Vault auth fail → auth method identify → credentials/role mapping → logs → fix → authenticate again.
Eng: I identify the authentication method and verify its credentials, role mapping, and recent configuration changes.
Example:
vault auth list → enabled authentication methods chudu.
vault token lookup → token valid aa check cheyyi.
Vault audit logs → authentication failure reason chudu.
Kubernetes auth ayithe → ServiceAccount/role mapping verify cheyyi.
AppRole ayithe → role-id/secret-id configuration check cheyyi.
Issue fix chesi → application authentication retest cheyyi.

---

3. Vault is returning "permission denied" while reading a secret. What will you check?
Answer:
First, I verify the exact secret path the application is requesting. Then I check which Vault policy is attached to the application's identity and whether that policy allows the required read capability on that path.
I avoid giving broad permissions and grant only the required access.
Simple ga gurthupettuko:
Permission denied → secret path → identity → policy → read permission → minimum access.
Eng: I verify the requested path and attached policy, then grant only the required read permission.
Example:
vault token lookup → current identity/token chudu.
vault policy read <policy> → allowed paths and capabilities check cheyyi.
vault kv get <path> → access test cheyyi.
Secret path mismatch ayithe → correct path use cheyyi.
Policy missing ayithe → required read permission add chesi → retest cheyyi.

---

4. Vault token is expiring frequently. What will you investigate?
Answer:
First, I check the token TTL, maximum TTL, renewal settings, and authentication method. Then I verify whether the application is renewing the token correctly.
If the token is intentionally short-lived, I check whether the application has proper token renewal or re-authentication logic.
Simple ga gurthupettuko:
Token expiry → TTL → max TTL → renewal → application behavior check.
Eng: I check token TTL and renewal behavior and verify that the application can renew or re-authenticate correctly.
Example:
vault token lookup → TTL and renewable status chudu.
Vault auth configuration → token TTL settings verify cheyyi.
Application logs → token renewal failures chudu.
Vault audit logs → authentication/renewal events inspect cheyyi.
Renewal configuration fix chesi → token lifetime behavior verify cheyyi.

---

5. Vault is sealed after a server restart. What will you check?
Answer:
First, I confirm that Vault is actually sealed and check the Vault server status. Then I identify the configured seal/unseal

mechanism and follow the organization's recovery procedure.
I also check Vault logs and storage/backend health before performing any recovery action.
Simple ga gurthupettuko:
Vault restart → sealed check → backend/logs → seal mechanism → approved unseal/recovery process.
Eng: I verify the sealed state, inspect Vault and backend health, and follow the configured recovery procedure.
Example:
vault status → sealed status chudu.
systemctl status vault → Vault service check cheyyi.
journalctl -u vault -n 100 --no-pager → startup errors chudu.
Vault storage/backend health verify cheyyi.
Configured auto-unseal/manual-unseal procedure follow chesi → Vault healthy state verify cheyyi.

Packer
6. Packer image build is failing during provisioning. What will you check?
Answer:
First, I identify the exact provisioning step that failed from the Packer output. Then I check the provisioning script, package installation, operating-system compatibility, network access, and required permissions.
I reproduce the failed command manually on a similar instance when necessary and fix the provisioning issue.
Simple ga gurthupettuko:
Packer fail → failed provision step → script → OS/package/network/permission → fix → rebuild.
Eng: I identify the failed provisioning step and validate the command, OS compatibility, network, and permissions.
Example:
packer build image.pkr.hcl → build output chudu.
Failed provisioner step identify cheyyi.
Provisioning script → manually failed command test cheyyi.
sudo/package permissions check cheyyi.
curl/apt/yum connectivity verify cheyyi.
Fix chesi → packer build image.pkr.hcl → image creation verify cheyyi.

---

7. Packer cannot connect to the EC2 instance during image creation. What will you check?
Answer:
First, I check whether the instance was created successfully and whether Packer can establish the configured SSH or WinRM connection.
Then I verify security groups, subnet routing, key configuration, username, public/private connectivity, and whether the instance is ready to accept connections.
Simple ga gurthupettuko:
Packer SSH fail → instance → network → SG → key/user → SSH readiness check.
Eng: I verify instance readiness, network access, security rules, credentials, and the configured connection method.
Example:
AWS EC2 → instance state check cheyyi.
nc -vz <instance-ip> 22 → SSH port connectivity test cheyyi.
ssh -i <key> <user>@<ip> → manual SSH test cheyyi.
Security Group → port 22 access check cheyyi.
Packer ssh_username and key configuration verify cheyyi.
Issue fix chesi → Packer build rerun cheyyi.

---

8. Packer creates an AMI successfully, but the application does not start from it. What will you investigate?
Answer:
First, I launch an instance from the generated AMI and check the application service and boot logs. Then I verify systemd configuration, startup scripts, environment variables, files, permissions, and required dependencies.
I compare the generated image with the working base instance to identify what was missing during image creation.
Simple ga gurthupettuko:
AMI created but app fail → launch instance → service/logs → config/dependencies → compare working image.
Eng: I troubleshoot the launched instance rather than assuming that successful AMI creation means the application is correctly configured.
Example:
systemctl status <service> → application service status chudu.
journalctl -u <service> -n 100 --no-pager → service errors inspect cheyyi.
journalctl -b → boot errors chudu.
Application config/env files verify cheyyi.
AMI vs working instance configuration compare chesi → image build fix chesi → new AMI test cheyyi.

AWS Deep Troubleshooting
9. An EC2 instance is running, but you cannot connect to it through SSH. What will you check?
Answer:
First, I verify the instance state, IP address, and reachability. Then I check the security group, subnet route table, network ACL, instance firewall, and SSH service.
I also verify that I am using the correct username and private key.
Simple ga gurthupettuko:
EC2 SSH fail → instance/IP → SG → NACL/route → OS firewall → SSH service → key/user.
Eng: I troubleshoot connectivity from the network layer through the OS and SSH service.

Example:
AWS EC2 → instance state and IP check cheyyi.
nc -vz <public-ip> 22 → TCP 22 test cheyyi.
Security Group → inbound SSH rule verify cheyyi.
Route Table → internet/NAT path check cheyyi.
NACL → inbound/outbound rules check cheyyi.
systemctl status sshd → SSH service verify cheyyi.
Correct key/user tho SSH test cheyyi.

---

10. EC2 instance has very high CPU utilization. How will you troubleshoot it?
Answer:
First, I identify which process is consuming CPU and correlate it with CloudWatch metrics and application behavior. Then I check recent deployments, traffic increase, scheduled jobs, and runaway processes.
I fix the root cause and scale the instance only if the workload genuinely requires more capacity.
Simple ga gurthupettuko:
CPU high → process identify → CloudWatch → traffic/deployment/job check → root cause fix → scale if required.
Eng: I identify the process and reason for CPU consumption before simply increasing the instance size.
Example:
CloudWatch → EC2 CPUUtilization graph chudu.
top → CPU-consuming process identify cheyyi.
ps aux --sort=-%cpu | head → top processes chudu.
Application logs → traffic/errors check cheyyi.
crontab -l → scheduled jobs check cheyyi.
Root cause fix chesi → CPU normal range ki vachindo CloudWatch lo verify cheyyi.

---

11. EC2 instance suddenly became unreachable after a network change. What will you check?
Answer:
First, I compare the current network configuration with the previous working configuration. Then I check route tables, security groups, NACLs, subnet configuration, and network interfaces.
I identify which network control changed and restore or correct only the required configuration.
Simple ga gurthupettuko:
Network change → old vs new compare → route/SG/NACL/subnet/ENI → changed setting identify → fix.
Eng: I compare the working and current network configuration to identify the change that caused the connectivity issue.
Example:
AWS VPC → Route Table → route changes compare cheyyi.
AWS EC2 → Security Group rules inspect cheyyi.
VPC → Network ACL rules check cheyyi.
EC2 → Network Interfaces inspect cheyyi.
ip route → OS route verify cheyyi.
Changed rule/config correct chesi → connectivity retest cheyyi.

---

12. An application in a private subnet cannot access the internet. What will you check?
Answer:
First, I verify that the private subnet route table has a default route pointing to a NAT Gateway or another approved egress path. Then I check the NAT Gateway, public subnet route, Internet Gateway, security groups, NACLs, and DNS configuration.
Simple ga gurthupettuko:
Private subnet internet fail → route → NAT → public subnet → IGW → SG/NACL → DNS.
Eng: I verify the complete egress path from the private subnet through the NAT Gateway.
Example:
AWS VPC → Private subnet Route Table → 0.0.0.0/0 target check cheyyi.
NAT Gateway → state and subnet verify cheyyi.
Public subnet Route Table → Internet Gateway route check cheyyi.
Security Group → outbound rules check cheyyi.
NACL → outbound/inbound ephemeral ports verify cheyyi.
curl -I https://example.com → internet connectivity test cheyyi.

---

13. ALB is returning 502 for an application. What will you check?
Answer:
First, I check the target group health and ALB access logs if available. Then I verify whether the backend application is listening on the configured target port and whether the health check path works.
I also check security groups between the ALB and targets and investigate application-side connection failures.
Simple ga gurthupettuko:
ALB 502 → target health → target port → health check → SG → backend logs.
Eng: I verify the target group and backend connectivity before changing ALB settings.
Example:
AWS ALB → Target Groups → Target Health chudu.
curl http://<target-ip>:<port>/health → backend health test cheyyi.
ss -lntp → target port listening aa check cheyyi.
ALB Security Group → target Security Group access verify cheyyi.
Application logs → request/connection errors chudu.
Fix chesi → target healthy and ALB response verify cheyyi.

14. ALB targets are repeatedly becoming unhealthy. What will you investigate?
Answer:
First, I verify the exact health-check failure reason. Then I compare the health-check protocol, port, path, timeout, and expected status code with the application's actual behavior.
I also check application startup time, security groups, and whether the service is intermittently failing.
Simple ga gurthupettuko:
Targets unhealthy → exact reason → health check config → app response → SG → intermittent issue check.
Eng: I compare the ALB health check configuration with the backend's actual response and availability.
Example:
AWS Target Group → Health status → failure reason chudu.
curl -v http://<target-ip>:<port>/health → same endpoint test cheyyi.
Target Group → Health Check path/port/protocol verify cheyyi.
Security Group → ALB-to-target access check cheyyi.
Application logs → health-check requests/errors chudu.
Issue fix chesi → targets consistently healthy ga unnayo verify cheyyi.

---

15. Auto Scaling Group is launching instances continuously. What will you check?
Answer:
First, I check why the newly launched instances are being terminated or marked unhealthy. Then I inspect target group health, EC2 status checks, launch template configuration, user-data scripts, and scaling policies.
I identify the reason for the unhealthy instances before changing the desired capacity.
Simple ga gurthupettuko:
ASG instances loop → why unhealthy/terminate → target health → EC2 status → launch template/user-data → scaling policy.
Eng: I identify why instances are failing health checks or terminating before changing the Auto Scaling configuration.
Example:
AWS Auto Scaling → Activity History → launch/terminate reason chudu.
Target Group → unhealthy reason check cheyyi.
EC2 → Status Checks verify cheyyi.
Launch Template → AMI/SG/IAM/user-data inspect cheyyi.
EC2 console → system/application logs chudu.
Root cause fix chesi → ASG stable ga instances maintain chesthunda verify cheyyi.

---

16. EKS worker nodes are becoming NotReady. What will you check?
Answer:
First, I identify whether the issue affects one node or multiple nodes. Then I check node conditions, kubelet status, resource pressure, networking, disk space, and recent node or cluster changes.
I also verify whether the node can communicate with the EKS control plane and whether required AWS permissions are working.
Simple ga gurthupettuko:
EKS NotReady → node conditions → kubelet → CPU/memory/disk → network → IAM → control-plane connectivity.
Eng: I identify the node condition causing NotReady and check kubelet, resources, networking, and AWS integration.
Example:
kubectl get nodes → NotReady nodes identify cheyyi.
kubectl describe node <node> → Conditions/events chudu.
systemctl status kubelet → kubelet status check cheyyi.
df -h → disk pressure check cheyyi.
free -m → memory pressure check cheyyi.
EKS node IAM/network configuration verify chesi → node Ready ayyindo verify cheyyi.

Observability Deeper Troubleshooting
17. CPU alert fires, but the application appears healthy. What will you investigate?
Answer:
First, I verify the actual CPU metric and alert expression instead of assuming the alert is wrong. Then I correlate CPU with request rate, latency, errors, container restarts, and application behavior.
If the CPU increase is legitimate but harmless, I review the alert threshold and whether the alert should consider additional signals.
Simple ga gurthupettuko:
CPU alert → metric verify → traffic/latency/errors correlate → real issue or noisy alert identify → tune if needed.
Eng: I correlate CPU with other application signals before deciding whether the alert is a false positive.
Example:
Prometheus → CPU query manually run cheyyi.
Grafana → CPU + request rate + latency dashboards compare cheyyi.
kubectl top pods → Pod CPU usage chudu.
Application logs → errors/restarts check cheyyi.
Alert threshold too sensitive ayithe → rule tune chesi → alert behavior verify cheyyi.

---

18. Alerts are firing repeatedly for the same issue. What will you check?
Answer:
First, I determine whether the alert represents a real ongoing problem or alert noise. Then I check the alert expression, evaluation interval, for duration, grouping, deduplication, and notification policy.
I tune the alert only after confirming that the underlying signal is correct.

Simple ga gurthupettuko:
Repeated alerts → real issue aa noise aa → expression → for/grouping/dedup → notification policy → tune.
Eng: I verify the alert signal first and then reduce noise through proper alert duration, grouping, and routing.
Example:
Prometheus → Alerts → alert state inspect cheyyi.
Alert rule → expression and for duration check cheyyi.
Alertmanager → grouping/inhibition rules inspect cheyyi.
Grafana → Notification Policies check cheyyi.
Repeated notification reason identify chesi → appropriate tuning chesi → test cheyyi.

---

19. Application is slow, but CPU and memory look normal. How will you troubleshoot?
Answer:
I do not assume the application is healthy just because CPU and memory are normal. I check request latency, database latency, external API calls, network latency, disk I/O, connection pools, and application traces.
I correlate the slow requests across metrics, logs, and traces to identify the actual bottleneck.
Simple ga gurthupettuko:
CPU/memory normal but app slow → latency → DB/API/network/I/O → logs + metrics + traces correlate.
Eng: I use multiple observability signals to identify latency outside CPU and memory.
Example:
Grafana → request latency/P95/P99 check cheyyi.
Prometheus → application latency metrics chudu.
Application logs → slow requests identify cheyyi.
Database metrics → query latency check cheyyi.
Tracing system → slow service/span identify cheyyi.
Bottleneck fix chesi → latency improvement verify cheyyi.

---

20. Metrics show an incident started at 10:00, but logs show errors from 09:55. How will you investigate?
Answer:
First, I verify whether the metrics and logs use the same timezone and timestamps. Then I check ingestion delays, scrape intervals, log shipping delays, buffering, and clock synchronization.
I normalize the timeline across systems before identifying the actual incident start time.
Simple ga gurthupettuko:
Metrics/log time mismatch → timezone → ingestion delay → scrape/log shipping → clock sync → timeline correct cheyyi.
Eng: I normalize timestamps and account for collection and ingestion delays before correlating the incident.
Example:
date → server time check cheyyi.
timedatectl → timezone/NTP status chudu.
Prometheus → scrape interval/data timestamp verify cheyyi.
Log collector → ingestion delay check cheyyi.
Grafana → dashboard timezone verify cheyyi.
Common timeline create chesi → actual incident sequence identify cheyyi.

---

21. Prometheus has metrics, but logs for the same application are missing. What will you check?
Answer:
Since metrics are available, I know the application is at least reachable by the monitoring system. I then focus on the logging pipeline: log generation, file/container stdout, log collector, permissions, parsing, labels, and destination storage.
I trace one application log from source to the final logging system.
Simple ga gurthupettuko:
Metrics present, logs missing → app logs generate avutunnaya → collector → parsing → labels → storage.
Eng: I trace the logging pipeline from the application to the final log backend.
Example:
kubectl logs <pod> → container logs generate avutunnaya check cheyyi.
Log collector status → Fluent Bit/Fluentd/agent check cheyyi.
Collector logs → shipping errors chudu.
Log backend → labels/index/query verify cheyyi.
Permissions/network check chesi → end-to-end log flow verify cheyyi.

---

22. Monitoring shows many alerts during every deployment. How will you troubleshoot the alert noise?
Answer:
First, I identify whether the alerts are caused by expected deployment behavior such as pod restarts, temporary latency, or reduced capacity. Then I correlate alerts with deployment events and check alert thresholds and evaluation periods.
I tune alerts to detect real customer-impacting conditions without hiding genuine failures.
Simple ga gurthupettuko:
Deployment time alerts → expected temporary behavior aa → correlate deployment → threshold/for/grouping tune → real failures preserve.
Eng: I correlate alerts with deployment behavior and tune them to avoid expected transient noise while preserving real incidents.
Example:
Deployment timeline → deployment start/end time note cheyyi.
Grafana → alerts + deployment events compare cheyyi.
Prometheus → alert expression inspect cheyyi.

Alert for duration and threshold check cheyyi.
Pod rollout behavior verify cheyyi.
Tuning tarvatha → deployment test chesi → unnecessary alerts reduce ayyayo verify cheyyi.

---

23. SLI looks healthy, but users are reporting problems. What will you investigate?
Answer:
First, I verify whether the selected SLI represents the actual user experience. Then I check whether the affected users, region, endpoint, tenant, or workflow is hidden by an aggregate metric.
I break the metric down by relevant dimensions and correlate it with logs, traces, and business-level signals.
Simple ga gurthupettuko:
SLI healthy but users complain → SLI correct aa → aggregate masking issue aa → segment by endpoint/region/user flow → logs/traces check.
Eng: I verify that the SLI actually represents the affected user experience and investigate segmented data.
Example:
Grafana → overall latency vs endpoint-level latency compare cheyyi.
Prometheus → labels by service/region/status code inspect cheyyi.
Logs → affected endpoint/user workflow identify cheyyi.
Tracing → affected request path inspect cheyyi.
Correct SLI define/tune chesi → user impact verify cheyyi.

---

24. Service has good availability but poor latency. What does this tell you?
Answer:
It means the service is mostly responding successfully, but responses are taking too long. I investigate latency independently from availability by checking P95/P99 latency, database performance, external dependencies, CPU throttling, network delays, and application traces.
I then identify the slow component and optimize it.
Simple ga gurthupettuko:
Availability good, latency bad → requests success avuthunnayi but slow → P95/P99 → DB/API/network/app trace.
Eng: Availability and latency are different reliability signals, so I investigate response time separately from success rate.
Example:
Grafana → P95/P99 latency chudu.
Prometheus → request duration metrics query cheyyi.
Application logs → slow requests identify cheyyi.
Database → slow queries check cheyyi.
Tracing → highest-latency service/span identify cheyyi.
Bottleneck fix chesi → latency metrics improve ayyayo verify cheyyi.

---

25. After fixing an incident, monitoring looks normal but users still report failures. What will you do?
Answer:
I do not close the incident immediately. First, I reproduce the user issue and check application logs, traces, endpoint-level metrics, load balancer behavior, DNS, and regional differences.
I verify the exact user journey instead of relying only on infrastructure-level health metrics.
Simple ga gurthupettuko:
Monitoring green but users fail → reproduce → endpoint/user journey → logs/traces/LB/DNS/region check → confirm recovery.
Eng: I validate recovery from the user's perspective rather than relying only on infrastructure dashboards.
Example:
curl -v https://<endpoint> → endpoint response test cheyyi.
ALB/Nginx logs → request status inspect cheyyi.
Application logs → request failure identify cheyyi.
Tracing → failed user request path inspect cheyyi.
DNS → dig <hostname> verify cheyyi.
Affected region/workflow test chesi → real user impact resolved ani confirm cheyyi.